

Student Name: Atharva Mohite

Student Session: cs6220

CS 6220 Big Data Systems, Fall 2026 · Homework 2, programming option

CS 6220 BIG DATA SYSTEMS · HOMEWORK 2 · PROGRAMMING OPTION

Five Word2Vec Models, Two Corpora, One Classifier

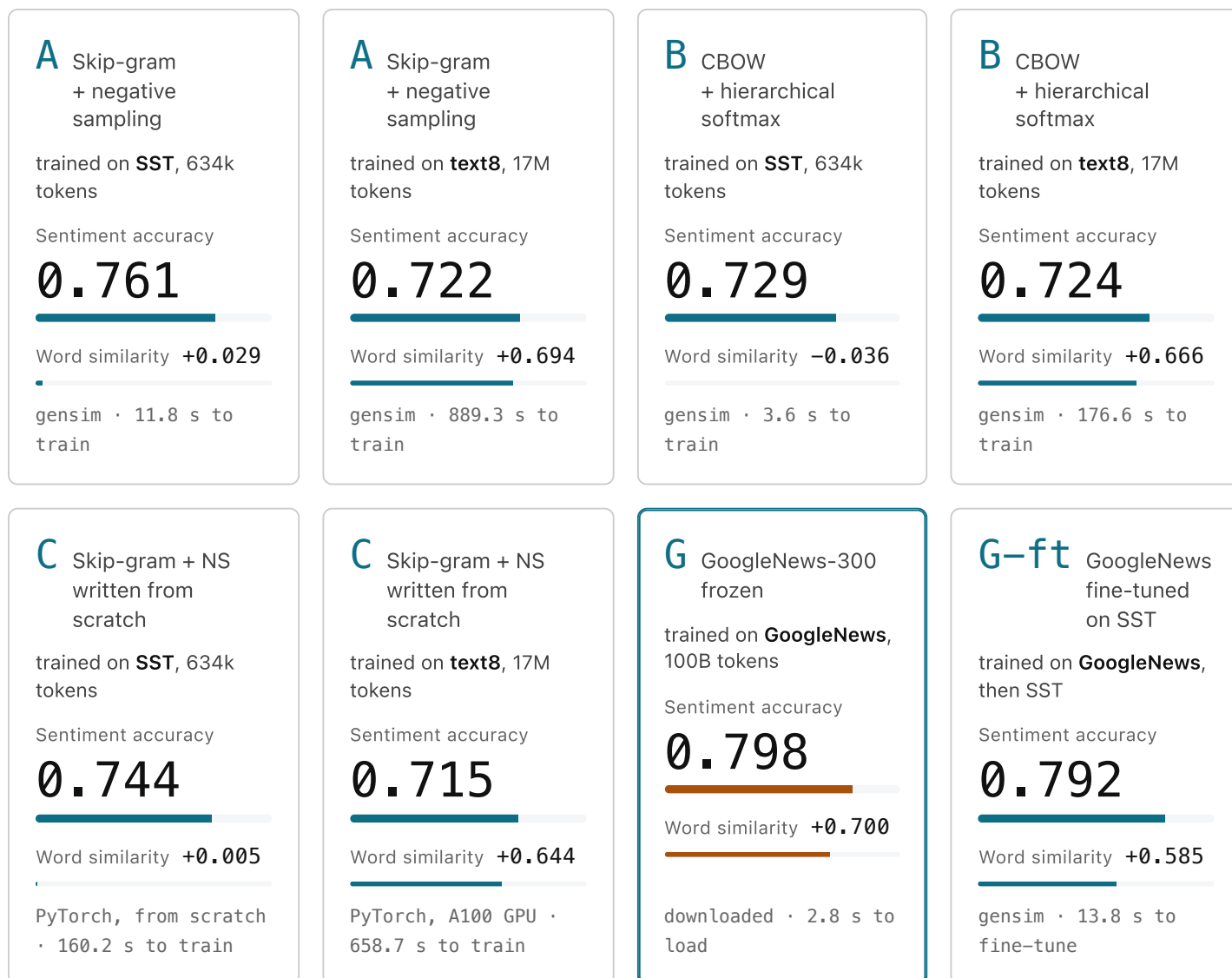
This experiment compares five routes to a Word2Vec model: two trained with gensim on movie reviews, a frozen pretrained model, a fine-tuned version of those pretrained weights, and a skip-gram implementation written from scratch. The three models trained from scratch are then retrained on text8, 17 million words of Wikipedia, to see how much the training text matters. The same pipeline measures sentiment accuracy, lexical structure, and runtime cost.

Corpora SST-2 phrases, 633,720 tokens; text8, 17,005,207 Vocabulary 14,309 (SST), 135,335 (text8)
Dimensions 300 Test split 1,821 sentences Machine M2 Air; PACE ICE A100 for C (text8)

WHERE THINGS STAND

Eight embeddings, one classifier

A, B and C appear twice, once trained on SST and once retrained on text8. Every model goes through the same evaluation. I look up each word vector, average the vectors into one sentence representation, fit logistic regression, and test on the same held-out SST-2 sentences. The classifier is never tuned for an individual model. This keeps the comparison focused on the embeddings.



Sentiment accuracy is the share of the 1,821 SST-2 test sentences classified correctly. Word similarity is the Spearman correlation between the model's cosine similarities and human ratings on WordSim-353: 1 is perfect agreement, 0 is none.

The warm start needs its own control. Comparing A with G-ft looks natural, but it mixes two changes. G-ft also uses a lower learning rate (0.005 against 0.025) and fewer epochs (8 against 10). **A2** uses A's random initialization with G-ft's exact schedule, leaving the starting weights as the only difference. A2 reaches 0.664, while G-ft reaches 0.792. On that controlled comparison, the warm start adds **+0.127**, not the +0.031 suggested by A versus G-ft. A's more aggressive schedule partly makes up for its random start, so the loose comparison understates the benefit.

MODEL	ARCHITECTURE	LOSS	TRAINED ON	ACCURACY	F1	OOV
A (SST)	Skip-gram	Negative sampling, k=10	633,720 tok	0.7611	0.7757	0.066
A (text8)	Skip-gram	Negative sampling, k=10	17,005,207 tok	0.7221	0.7413	0.169
B (SST)	CBOW	Hierarchical softmax	633,720 tok	0.7293	0.7501	0.066
B (text8)	CBOW	Hierarchical softmax	17,005,207 tok	0.7243	0.7490	0.169
C (SST)	Skip-gram	Negative sampling, k=10	633,720 tok	0.7441	0.7610	0.066
C (text8)	Skip-gram	Negative sampling, k=10	17,005,207 tok	0.7150	0.7340	0.169
G	Skip-gram	Negative sampling, k=5	100,000,000,000 tok	0.7979	0.8107	0.271
G-ft	Skip-gram	Negative sampling, k=10	633,720 tok	0.7919	0.8089	0.066
A2	Skip-gram	Negative sampling, k=10	633,720 tok	0.6645	0.7070	0.066

A2 is a control, not a sixth model. Its 0.664 score is the lowest in the table, but that result is useful: it makes the claim about G-ft's warm start testable. The three text8 rows are A, B and C trained on Wikipedia instead of SST. [More training text](#) covers them.

DELIVERABLE 3

Five test examples, ranked by the five main models

These examples show the held-out sentiment task at sentence level. For each model, the table gives the classifier's two labels with their probabilities and the top-5 closest *training* sentences by cosine similarity. Neighbour agreement is the fraction of those retrieved sentences whose gold label matches the test example. It gives a small, concrete view of what evidence sits nearby in the embedding space.

I selected all five by rule, not by eye. The rule appears above each example. Accuracy within this small set is A 3/5, B 3/5, C 3/5, G 4/5, G-ft 3/5.

EXAMPLE 1 · PLAIN POSITIVE

this is one of polanski 's best films .

gold label: **positive**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A (SST)	positive 0.838	negative 0.162	correct	1.0
B (SST)	positive 0.694	negative 0.306	correct	1.0
C (SST)	positive 0.656	negative 0.344	correct	1.0
G	positive 0.855	negative 0.145	correct	1.0
G-ft	positive 0.808	negative 0.192	correct	1.0

G-ft
retrieves

this is one of the year 's best films . · 0.99

is one of the year 's best films . · 0.98

is one of the year 's best films · 0.97

Training sentences are ordered by cosine similarity. A **green tint** means the retrieved sentence's label matches the test sentence's gold label. This is separate from the gold tint in the neighbour tables.

EXAMPLE 2 · PLAIN NEGATIVE

arnold 's jump from little screen to big will leave frowns on more than a few faces .

gold label: **negative**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A (SST)	positive 0.593	negative 0.407	wrong	1.0
B (SST)	negative 0.534	positive 0.466	correct	1.0
C (SST)	negative 0.546	positive 0.454	correct	0.8
G	negative 0.668	positive 0.332	correct	0.0
G-ft	positive 0.512	negative 0.488	wrong	1.0

G-ft
retrieves

- big fat liar is little more than home alone raised to a new , self-deprecating l · 0.93
- is little more than home alone raised to a new , self-deprecating level . · 0.92
- the film runs on a little longer than it needs to · 0.92

Training sentences are ordered by cosine similarity. A **green tint** means the retrieved sentence's label matches the test sentence's gold label. This is separate from the gold tint in the neighbour tables.

EXAMPLE 3 · POSITIVE DESPITE NEGATION

finally , a genre movie that delivers -- in a couple of genres , no less .

gold label: **positive**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A (SST)	negative 0.596	positive 0.404	wrong	0.4
B (SST)	negative 0.643	positive 0.357	wrong	0.2
C (SST)	negative 0.597	positive 0.403	wrong	0.4
G	positive 0.680	negative 0.320	correct	0.4
G-ft	negative 0.508	positive 0.492	wrong	0.8

G-ft
retrieves

neither quite a comedy nor a romance , more of an impish divertissement of theme · 0.95

singer/composer bryan adams contributes a slew of songs -- a few potential hits · 0.94

singer/composer bryan adams contributes a slew of songs -- a few potential hits · 0.94

Training sentences are ordered by cosine similarity. A **green tint** means the retrieved sentence's label matches the test sentence's gold label. This is separate from the gold tint in the neighbour tables.

EXAMPLE 4 · CONTRASTIVE CLAUSE

but what saves lives on the freeway does not necessarily make for persuasive viewing .

gold label: **negative**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A (SST)	negative 0.592	positive 0.408	correct	1.0
B (SST)	positive 0.574	negative 0.426	wrong	0.8
C (SST)	positive 0.614	negative 0.386	wrong	0.6
G	positive 0.565	negative 0.435	wrong	0.8
G-ft	negative 0.502	positive 0.498	correct	0.8

G-ft
retrieves

does n't show enough of the creative process or even of what was created for the · 0.92

if swimfan does catch on , it may be because teens are looking for something to · 0.92

what might have been acceptable on the printed page of iles ' book does not tran · 0.92

Training sentences are ordered by cosine similarity. A **green tint** means the retrieved sentence's label matches the test sentence's gold label. This is separate from the gold tint in the neighbour tables.

EXAMPLE 5 · NEGATIVE WITH STACKED NEGATION

no movement , no yuks , not much of anything .

gold label: **negative**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A (SST)	negative 0.972	positive 0.028	correct	1.0
B (SST)	negative 0.955	positive 0.045	correct	1.0
C (SST)	negative 0.860	positive 0.140	correct	0.6
G	negative 0.964	positive 0.036	correct	1.0
G-ft	negative 0.989	positive 0.011	correct	1.0

G-ft
retrieves

... no charm , no laughs , no fun , no reason to watch . · 0.92

no charm , no laughs , no fun , no reason to watch . · 0.92

no charm , no laughs , no fun , no reason to watch . · 0.91

Training sentences are ordered by cosine similarity. A **green tint** means the retrieved sentence's label matches the test sentence's gold label. This is separate from the gold tint in the neighbour tables.

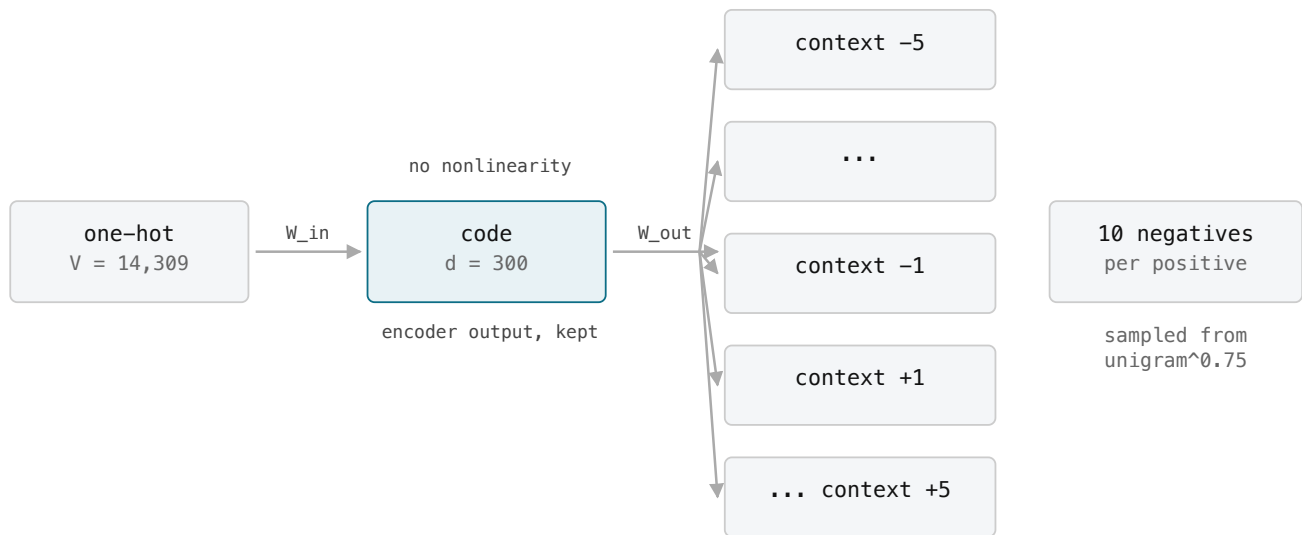
Example 3 separates GoogleNews from every model trained on SST. In this sentence, `no less` is an intensifier rather than a negation. Of the five main models, only the frozen GoogleNews model reads it that way. Every SST-trained model, including G-ft, predicts negative.

Example 5 deserves a closer look. All five main models get it right with high confidence, and retrieval shows why. The nearest training sentence is `no charm , no laughs , no fun`, a near-duplicate of the test sentence's stacked-negation pattern. The models succeed through lexical overlap, not by composing the negation. The same shortcut fails on Example 3 but happens to work here.

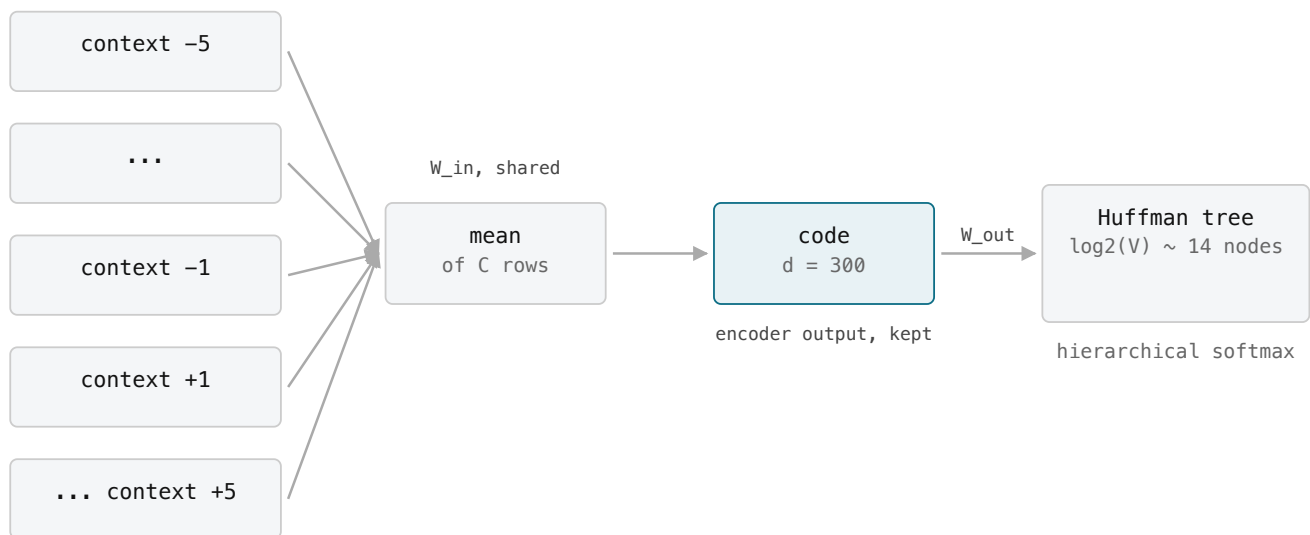
DELIVERABLE 1

How the two architectures work

A Word2Vec model comes down to two matrices and a dot product. It has no hidden layer or nonlinearity. `w_in` stores one row per word and becomes the final embedding table. `w_out` is used for the training objective, then discarded.



A and C use skip-gram with negative sampling. A centre word predicts each surrounding word independently. Each observed pair is scored against 10 negative samples from the unigram distribution raised to 0.75, avoiding a softmax over all 14,309 words.



B uses CBOW with hierarchical softmax. The context rows are averaged into one vector, which predicts the centre word. A binary tree over the vocabulary reduces one prediction from V decisions to $\log_2(V)$.

The encoder, code, and decoder analogy only goes so far. The terms come from autoencoders, which reconstruct their own input. Word2Vec predicts a *neighbouring* word and throws away its output matrix after training. In the mapping above, the one-hot word is the input, `W_in` is the encoder, its 300-dimensional row is the code, `W_out` is the decoder, and the predicted context word is the output.

DELIVERABLE 2

Training and test data

The sentiment data comes from SST-2, but its two releases cannot be swapped freely. GLUE replaces every test label with -1 because the split is reserved for its leaderboard. I use the labelled SetFit test split instead. `src/data.py` raises an error unless both classes appear in the loaded test labels.

SPLIT	SOURCE	ROWS	TOKENS	MEAN LENGTH	POSITIVE	USED FOR
train	stanfordnlp/sst2 (phrase level)	67,348	633,720	9.4	55.8%	embedding training and classifier fitting
dev	SetFit/sst2	872	17,046	19.5	50.9%	held out, unused
test	SetFit/sst2	1,821	35,023	19.2	49.9%	every number on this page

14,309

vocabulary at min_count=2, phrase-level train

7,140

vocabulary at min_count=2, sentence-level train

6.6%

test-token OOV rate, SST-trained vocabulary

27.1%

test-token OOV rate, GoogleNews-300 vocabulary

What the rows look like

Deliverable 2b asks for five training examples and four test examples, two positive and two negative. Each example was selected by a stated rule rather than picked by eye.

FIVE TRAINING EXAMPLES

SELECTED FOR	TEXT	GOLD LABEL
positive, plain	that loves its characters and communicates something rather beautiful about human nature	positive
negative, plain	remains utterly satisfied to remain the same throughout	negative
negation	for those moviegoers who complain that ` they do n't make movies like they used to anymore	negative
contrastive clause	the plot is nothing but boilerplate clichés from start to finish ,	negative
film-domain vocabulary	, good action , good acting , good dialogue , good pace , good cinematography .	positive

FOUR TEST EXAMPLES

SELECTED FOR	TEXT	GOLD LABEL
positive	this is one of polanski 's best films .	positive
positive	finally , a genre movie that delivers -- in a couple of genres , no less .	positive
negative	arnold 's jump from little screen to big will leave frowns on more than a few faces .	negative
negative	no movement , no yuks , not much of anything .	negative

The leakage check found one duplicate. The phrase rows are labelled treebank subtrees taken only from the training split. An exact string check still found the fragment `what 's next ?` in both training and test data. I removed it. There was no overlap with dev, and the training count fell from 67,349 to 67,348 rows.

Most of the phrase-level text repeats the sentences. 97.5% of the phrase rows (65,640 of 67,348) appear word for word inside the 6,920 training sentences, because each row is a subtree of one of them. The phrase corpus has 4.75× the tokens of the sentence release, but most of those tokens are the same text cut into overlapping pieces. The repetition also doubles the vocabulary. At min_count=2 it grows from 7,140 to 14,309 words, and 7,132 of the 7,184 added words occur only once in the sentences. They pass the cutoff because the fragments of that one sentence repeat them. [Vector arithmetic](#) shows what this does to the embeddings.

SECOND TRAINING CORPUS: TEXT8

A, B and C were also trained on text8, the first 100 MB of an English Wikipedia dump cleaned by Matt Mahoney (mattmahoney.net/dc/text8.zip). It has 17,005,207 tokens and a vocabulary of 135,335 words at min_count=2. The text contains only lowercase letters and spaces. Punctuation is removed and numbers are spelled out digit by digit, so `1989` becomes `one nine eight nine`. text8 is used only to train word vectors; the classifier and every test stay on SST. The file is downloaded by `src/run_text8.py` rather than stored in the repository.

The pretrained baseline is GoogleNews-300, a skip-gram model with negative sampling trained on roughly 100 billion news tokens. It comes from code.google.com/archive/p/word2vec/, is loaded through gensim-data, and is capped at the 500,000 most frequent words. I did not use GloVe as the pretrained baseline because it factorizes a co-occurrence matrix rather than training Word2Vec. That would change the question being tested.

PRETRAINED MODEL, AS DELIVERABLE 2A ASKS

FIELD	VALUE
Model	GoogleNews-vectors-negative300
Reference	Mikolov, Sutskever, Chen, Corrado and Dean (2013), <i>Distributed Representations of Words and Phrases and their Compositionality</i> , NIPS. arxiv.org/abs/1310.4546
Distribution URL	code.google.com/archive/p/word2vec/ , fetched through gensim-data as <code>word2vec-google-news-300</code>
Training corpus	Google News, roughly 100 billion tokens. The corpus was not released, so there is no public corpus link.
Architecture	Skip-gram with negative sampling, 300 dimensions, 3M words and phrases
Test set used here	Google analogy set (<code>questions-words.txt</code>), 19,544 questions, shipped in <code>data/eval/</code>
Accuracy, measured here	0.7561 across 15,434 attempted questions (79.0% coverage, with the candidate pool limited to the top 50,000 words)
Accuracy, as published	Not quoted. The Google Code archive does not expose figures in a form that can be fetched, and its linked papers report results for the models studied in those papers, not this released file. I report the measured result above with its exact conditions instead of attaching an unverified published number to the model.

DELIVERABLE 5

Hyperparameters

A and B intentionally change two things together: the architecture and the loss. All other settings stay fixed, so the comparison is between the two standard Word2Vec configurations. The sweep below separates the two changes.

	A (SST)	B (SST)	C (SST)	G	G-FT	A2
Architecture (sg)	1	0	1	1	1	1
Hierarchical softmax	0	1	0	0	0	0
Negatives	10	0	10	5	10	10
Dimensions	300	300	300	300	300	300
Window	5	5	5	5	5	5
Min count	2	2	2	5	2	2
Epochs	10	10	10	None	8	8
Initial LR	0.025	0.025	0.001	0.025	0.005	0.005
Subsample threshold	0.001	0.001	0.001	1e-05	0.001	0.001

SWEEP

Each setting below was trained 3 times on the SST phrase corpus, with seeds 42, 43, 44. The tables give the mean and standard deviation across seeds. One factor changes at a time, and everything else stays at A's or B's settings. Training time is the mean wall clock on the M2 Air.

Architecture and loss

SETTING	SST-2 ACCURACY	WORDSIM-353	MEN-3K	TRAINING TIME
skip-gram + NS	0.752 ± 0.006	+0.008 ± 0.018	+0.122 ± 0.008	15.2 ± 0.9 s
skip-gram + HS	0.752 ± 0.006	+0.043 ± 0.014	+0.042 ± 0.006	13.7 ± 0.5 s
CBOW + NS	0.735 ± 0.005	-0.005 ± 0.019	+0.128 ± 0.004	5.8 ± 0.1 s
CBOW + HS	0.737 ± 0.005	-0.002 ± 0.018	-0.000 ± 0.004	6.3 ± 0.6 s

Context window

SETTING	SST-2 ACCURACY	WORDSIM-353	MEN-3K	TRAINING TIME
A, window 2	0.738 ± 0.002	+0.028 ± 0.008	+0.130 ± 0.002	11.2 ± 0.4 s
A, window 5	0.752 ± 0.006	+0.008 ± 0.018	+0.122 ± 0.008	15.2 ± 0.9 s
A, window 10	0.755 ± 0.005	+0.029 ± 0.020	+0.095 ± 0.002	26.1 ± 1.7 s
B, window 2	0.715 ± 0.002	-0.006 ± 0.013	+0.010 ± 0.010	4.8 ± 0.1 s
B, window 5	0.737 ± 0.005	-0.002 ± 0.018	-0.000 ± 0.004	6.3 ± 0.6 s
B, window 10	0.741 ± 0.007	-0.014 ± 0.010	+0.005 ± 0.008	7.1 ± 1.4 s

Vector dimensions

SETTING	SST-2 ACCURACY	WORDSIM-353	MEN-3K	TRAINING TIME
A, 50 dims	0.746 ± 0.006	+0.007 ± 0.006	+0.104 ± 0.009	6.6 ± 0.3 s
A, 100 dims	0.751 ± 0.007	+0.018 ± 0.019	+0.106 ± 0.009	8.8 ± 0.2 s
A, 300 dims	0.752 ± 0.006	+0.008 ± 0.018	+0.122 ± 0.008	15.2 ± 0.9 s
B, 50 dims	0.681 ± 0.004	-0.013 ± 0.012	+0.012 ± 0.007	2.8 ± 0.3 s
B, 100 dims	0.689 ± 0.003	+0.020 ± 0.026	+0.005 ± 0.003	3.2 ± 0.1 s
B, 300 dims	0.737 ± 0.005	-0.002 ± 0.018	-0.000 ± 0.004	6.3 ± 0.6 s

Negative samples

SETTING	SST-2 ACCURACY	WORDSIM-353	MEN-3K	TRAINING TIME
A, 5 negatives	0.761 ± 0.004	+0.008 ± 0.022	+0.121 ± 0.004	18.5 ± 4.7 s
A, 10 negatives	0.752 ± 0.006	+0.008 ± 0.018	+0.122 ± 0.008	15.2 ± 0.9 s
A, 20 negatives	0.760 ± 0.004	+0.018 ± 0.003	+0.125 ± 0.009	38.4 ± 6.7 s

Architecture matters and the loss does not. Skip-gram averages 0.752 with negative sampling and 0.752 with hierarchical softmax. CBOW averages 0.735 and 0.737. Changing the loss moves accuracy by at most 0.002, while changing the architecture moves it by about 0.015. The gap between A and B comes from skip-gram against CBOW.

Single runs overstate the A-B gap. The headline reports A at 0.761 and B at 0.729, a gap of 0.032. Over three seeds the same settings average 0.752 and 0.737, a gap of 0.015. Each headline figure is one run, and A's landed above its mean while B's landed below.

Wider windows help both models. Going from a window of 2 to 10 raises A from 0.738 to 0.755 and B from 0.715 to 0.741. For skip-gram it also more than doubles training time, from 11.2 to 26.1 seconds, because every extra context word is another prediction. **CBOW needs more dimensions than skip-gram.** B gains 0.056 going from 50 to 300 dimensions, while A gains only 0.006. **The number of negatives shows no clear trend.** 5, 10 and 20 negatives give 0.761, 0.752 and 0.760. The middle setting is the lowest, which fits no trend, and with three seeds a gap that small cannot be separated from seed noise. 20 negatives take 38 seconds.

No setting produces similarity structure. WordSim-353 stays between -0.014 and +0.043 across all 14 distinct settings. On SST, the hyperparameters shift sentiment accuracy by a few points, but none of them fixes similarity. Only more data did, as [More training text](#) shows.

Training times drift as the fanless M2 Air warms up. The 10-negative settings were the first runs of the sweep, on a cool machine, and came out faster than the 5-negative runs even though they do more work. Small time differences between settings are therefore not reliable.

DELIVERABLES 3 AND 5

Top-ranked neighbours for five queries

The five queries are meant to expose differences between the models: a polysemous noun, a sentiment adjective, a domain-sensitive word, a rare film term, and the most frequent word in English.

word 0.00

A **gold-tinted chip** appears only in row G-ft. It marks a neighbour that G does *not* return for the same query, so it entered the neighbourhood after fine-tuning on SST. All other chips are grey.

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A (SST)	twists 0.48	pawn 0.46 court 0.46 device 0.46 oh-so 0.45	nifty 0.45 cliché-riddled 0.44 stop-and-start 0.44 triteness 0.44 convenient 0.44
A (text8)	ridolfi 0.55	popish 0.51 gattaca 0.49 subplot 0.51 plots 0.49	babington 0.48 whodunit 0.48 counterpointed 0.48 replicants 0.48 dramatized 0.48
B (SST)	ending 0.32	accomplishment 0.28 sweetness 0.26 flourish 0.25 despair 0.25	flashy 0.25 clancy 0.25 mastering 0.25 compressed 0.24 sure-fire 0.24
B (text8)	storylines 0.37	storyline 0.36 gattaca 0.36 story 0.34 sequel 0.35	plots 0.32 villain 0.32 horror 0.31 character 0.32 flashback 0.32
C (SST)	twists 0.86	device 0.84 halfwit 0.83 oh-so 0.84 court 0.83	stop-and-start 0.81 maneuvers 0.81 holes 0.80 sillified 0.81 nurses 0.80
C (text8)	ridolfi 0.59	subplot 0.58 popish 0.56 storyline 0.55 plots 0.58	gattaca 0.55 conspirators 0.53 storylines 0.53 macguffin 0.52 fawkes 0.53
G	plots 0.76	Plot 0.65 plotted 0.60 plotline 0.59 plotting 0.63	plotters 0.54 conspiracies 0.52 conspiracy 0.51 storyline 0.53

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
G-ft	plots 0.77		foiled_plot 0.50
		plotting 0.68	conspiracies 0.51
		plotted 0.62	conspiracy 0.51
		plotline 0.56	oh-so 0.50
		storyline 0.52	conspirators 0.48
			subplots 0.47

terrible

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A (SST)	fitting 0.70	synergistic 0.67 stultifyingly 0.67 reconstruction 0.66 bloodwork 0.66	besotted 0.59 impulse 0.59 propelled 0.58 freaky 0.58 regard 0.57
A (text8)	horrible 0.55	terrifying 0.51 thulsa 0.51 irreparable 0.50 toils 0.50	betrayals 0.49 hitlerite 0.49 frailty 0.48 devastations 0.48 calamity 0.48
B (SST)	thrilling 0.40	chump 0.36 ballistic 0.34 motion 0.34 paint-by-numbers 0.33	view 0.33 derrida 0.33 ecks 0.33 cinderella 0.33 raises 0.33
B (text8)	disastrous 0.35	devastating 0.34 horrible 0.30 tragic 0.29 severe 0.29	unfortunate 0.29 appalling 0.29 danger 0.28 bad 0.28 lendl 0.28
C (SST)	+ 0.85	besotted 0.84 allegedly 0.83 unfulfilling 0.83 dungeons 0.83	phenomenal 0.82 lanes 0.82 retread 0.82 inspired 0.81 snl 0.81
C (text8)	horrible 0.72	horrific 0.71 terrifying 0.70 irreparable 0.69 misfortune 0.69	pestilence 0.68 calamity 0.68 wanton 0.67 sorrow 0.67 cruel 0.67
G	horrible 0.92	horrendous 0.85 dreadful 0.80 awful 0.75 horrid 0.72	atrocious 0.69 horrific 0.68 bad 0.68 appalling 0.68 horrible_horrible 0.67
G-ft	horrible 0.89	dreadful 0.80 awful 0.78	appalling 0.70 horrific 0.68

MODEL	TOP-1	TOP-2 TO 5		TOP-6 TO 10	
		atrocious 0.72	horrid 0.71	lousy 0.67	hideous 0.67
				clockstoppers 0.67	

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A (SST)	raving 0.59	trek 0.59 high-powered 0.58 pop-induced 0.57 jake 0.57	bestowed 0.57 anteing 0.56 wattage 0.56 lone 0.56 indomitability 0.55
A (text8)	trek 0.70	stars 0.64 eridani 0.56 spangled 0.55 procyon 0.55	duetted 0.54 ophiuchus 0.54 gazer 0.54 shatterpoint 0.54 teleporters 0.54
B (SST)	winners 0.33	kevin 0.31 splendid 0.30 has 0.30 acting 0.30	pulls 0.30 watstein 0.29 contributes 0.29 dustin 0.28 teaming 0.28
B (text8)	stars 0.57	attractiva 0.36 spoor 0.36 kzin 0.35 planet 0.35	galaxies 0.34 brightest 0.34 bedivere 0.34 mckellen 0.34 amias 0.33
C (SST)	jake 0.88	rising 0.87 gyllenhaal 0.85 charismatic 0.85 top-billed 0.82	pop-induced 0.77 willis 0.74 bestowed 0.73 high-powered 0.73 kevin 0.73
C (text8)	stars 0.64	trek 0.63 centauri 0.52 ophiuchus 0.50 prequels 0.49	brightest 0.49 procyon 0.49 shatterpoint 0.48 eclipsing 0.47 gazer 0.47
G	stars 0.78	superstar 0.73 starlet 0.64 megastar 0.62 heart_throb 0.57	heartthrob 0.54 superstars 0.53 standout 0.52 stardom 0.52 hunk 0.51
G-ft	superstar 0.75	stars 0.74 gyllenhaal 0.60	first-rate 0.58 standout 0.58 starring 0.56 huston 0.56

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
		top-billed 0.60	legend 0.55
		pop-induced 0.58	

cinematography

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A (SST)	rinzler 0.71	water-born 0.70 shimmering 0.70 hennings 0.69 lisa 0.69	l. 0.69 conrad 0.68 italicized 0.68 robotically 0.68 pitted 0.67
A (text8)	expressionistic 0.63	bafta 0.60 shazam 0.58 screenwriting 0.57 cine 0.57	cinematographer 0.57 filmmaking 0.57 elmes 0.57 rashomon 0.57 desaturated 0.56
B (SST)	color 0.38	gantz 0.38 beaches 0.37 mattei 0.37 hotels 0.36	improbability 0.36 acting 0.35 lumpen 0.35 procedural 0.35 reggio 0.34
B (text8)	film 0.41	horror 0.37 photography 0.36 oscar 0.36 films 0.36	director 0.35 picture 0.34 projector 0.33 vangelis 0.33 smeeton 0.33
C (SST)	hennings 0.90	water-born 0.88 dubbed 0.88 naturalistic 0.87 murky 0.87	shimmering 0.86 phenomenal 0.86 robotically 0.86 atrocious 0.84 acute 0.84
C (text8)	bafta 0.74	cine 0.73 versatility 0.73 elmes 0.73 directorial 0.73	vangelis 0.73 filmmaking 0.72 expressionistic 0.71 wuxia 0.70

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
			filmfare 0.70
G	camerawork 0.77	cinematographer 0.75 Cinematography 0.70 Cinematographer 0.67 cinematography_editing 0.63	cinematographers 0.63 Best_Cinematography 0.62 visual_effects 0.60 filmmaking 0.60 visuals 0.59
G-ft	camerawork 0.76	cinematographer 0.75 visuals 0.61 screenplay 0.59 photography 0.59	choreography 0.58 filmmaking 0.56 helmer 0.55 lensing 0.54 narration 0.54

the

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A (SST)	of 0.40	deleting 0.36 outward 0.35 tully 0.35 1987 0.35	unambitious 0.35 screenplays 0.35 hubristic 0.35 homeric 0.35 indulges 0.35
A (text8)	of 0.73	and 0.71 in 0.70 a 0.69 to 0.60	s 0.58 one 0.56 platforming 0.56 which 0.56 thq 0.55
B (SST)	our 0.22	his 0.21 current 0.19 dong 0.16 whose 0.15	german 0.15 of 0.15 my 0.15 in 0.14 art 0.14
B (text8)	a 0.44	its 0.42 this 0.37 another 0.35 their 0.31	s 0.29 an 0.29 our 0.28 his 0.28 lindesfarne 0.26
C (SST)	inside-show-biz 0.94	producing 0.93 pierce 0.93 silent 0.93 1790 0.93	savaged 0.93 democratic 0.93 shoulder 0.92 worlds 0.92 indulges 0.92
C (text8)	nonparticipation 0.63	xviith 0.63 maastrichtian 0.62 kingdomtide 0.62 jedda 0.62	jurua 0.62 latitudinal 0.62 of 0.62 centromedian 0.61 unpredicted 0.61
G	this 0.59	in 0.54 that 0.53 ofthe 0.52 another 0.47	however 0.47 one 0.47 entire 0.46 its 0.46 which 0.46
G-ft	of 0.63	. 0.59 2002 0.58 spielberg 0.54	salton 0.53 self-conscious 0.53

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
		entire 0.54	pacino 0.53 revenge-of-the-nerds 0.52 auteuil 0.52

G and G-ft behave differently from A, B and C trained on SST. G returns clear synonyms and inflections. G-ft preserves them while adding film-specific associations: `visuals`, `screenplay` and `photography` enter `cinematography`, and `storyline` enters `plot`. The three SST-trained models return almost noise for every query. Their 633,720-token corpus does not contain enough co-occurrence evidence to organize the words reliably. The [similarity benchmarks](#) measure that gap directly.

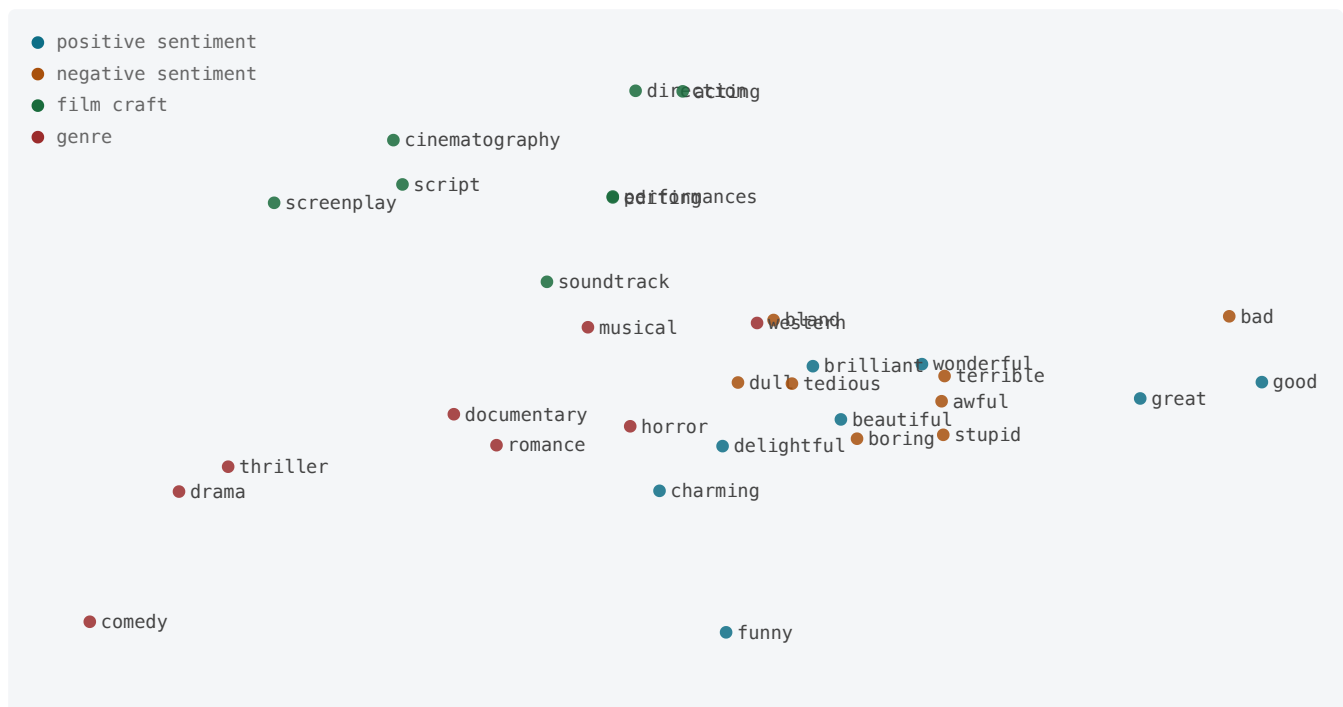
The text8 rows are A, B and C retrained on Wikipedia. With 17,005,207 tokens, their neighbours become sensible: `terrible` gives `horrible` for all three, and B's `plot` gives `storylines` and `storyline`. Some neighbours now come from encyclopedia topics rather than film reviews, such as `trek` for `star`. [More training text](#) compares the two corpora in full.

HANDOUT REQUIREMENT

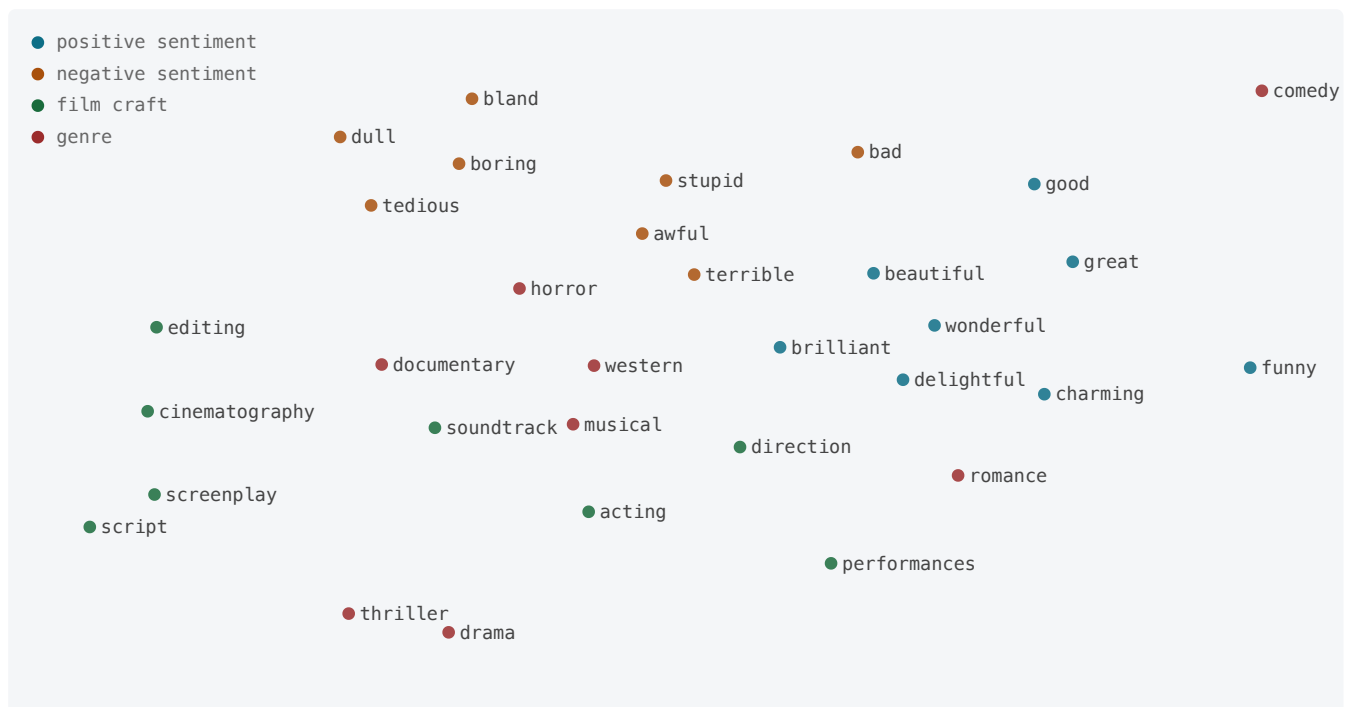
The vector space in two dimensions

The plotted words were fixed before seeing the result. They cover four groups: positive sentiment, negative sentiment, film craft, and genre. If an embedding carries useful semantic structure, words from the same group should remain close after projection.

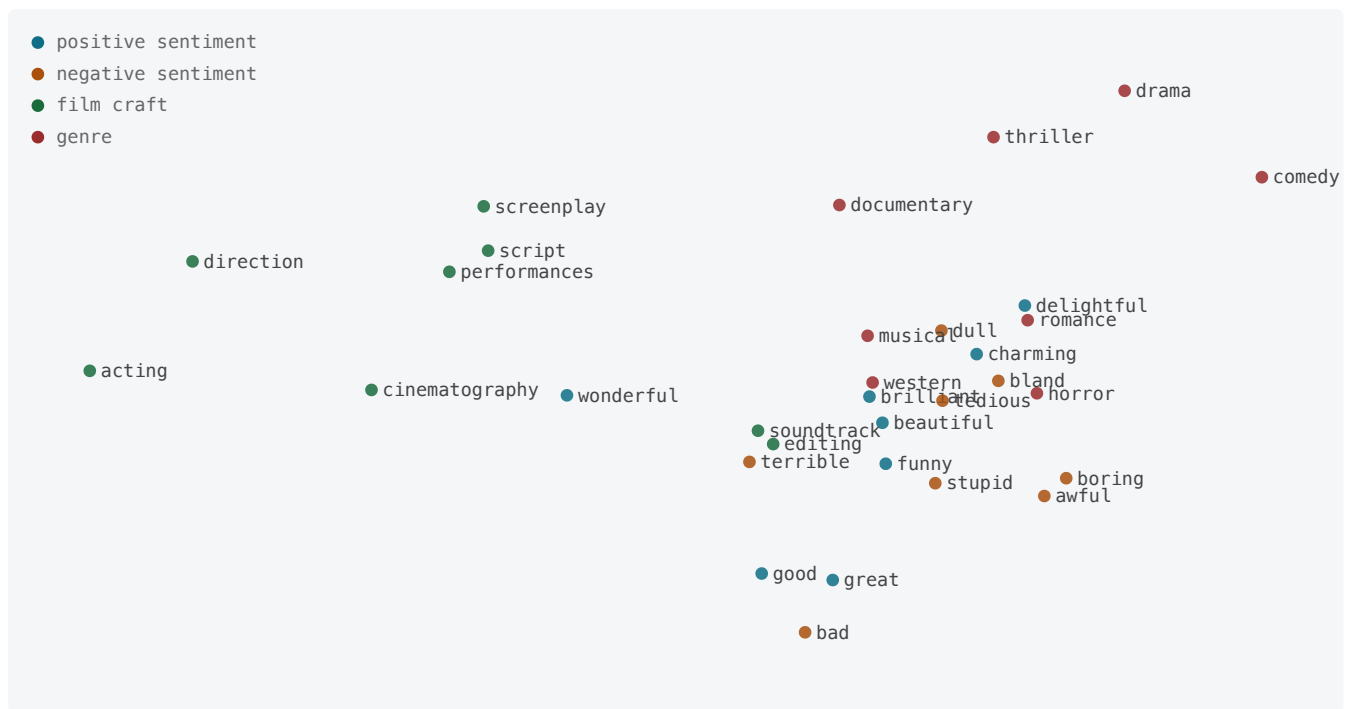
The scatter plots can be easy to over-read, so each one includes a silhouette score against the known group labels. A score near 0 means the groups overlap; higher scores indicate cleaner separation.



PCA, Model G-ft. The plot contains 32 words from four semantic groups fixed in advance, with explained variance 16.2% and 9.6%. Its two-dimensional silhouette score against those group labels is **0.164**.



t-SNE, Model G-ft. The plot contains 32 words from four semantic groups fixed in advance, with perplexity 10. Its two-dimensional silhouette score against those group labels is **0.187**.



PCA, Model A. The plot contains 32 words from four semantic groups fixed in advance, with explained variance 8.6% and 7.5%. Its two-dimensional silhouette score against those group labels is **0.036**.



t-SNE, Model A. The plot contains 32 words from four semantic groups fixed in advance, with perplexity 10. Its two-dimensional silhouette score against those group labels is **0.051**.

MODEL	PCA SILHOUETTE	T-SNE SILHOUETTE	WORDSIM-353	ANALOGY ACCURACY
A (SST)	+0.036	+0.051	+0.029	0.001
B (SST)	+0.035	-0.033	-0.036	0.002
C (SST)	-0.004	+0.110	+0.005	0.002
G	+0.120	+0.121	+0.700	0.756
G-ft	+0.164	+0.187	+0.585	0.675

The projections match the quantitative results. Under PCA, G-ft and G separate the four groups (+0.164 and +0.120), while A, B, and C do not (+0.036, +0.035, -0.004). WordSim-353 and the analogy benchmark produce the same ordering. Even so, the first two PCA components capture less than 20% of the variance for every model. Each picture is only a narrow view of a 300-dimensional space.

Vector arithmetic and analogies

The handout names `king - man + woman = queen` specifically. Below it are four queries built from film-review vocabulary. Every word in those four occurs in SST, so the SST-trained models cannot miss them for lack of vocabulary. The standard Google analogy benchmark follows.

king - man + woman expected `queen`

MODEL	TOP ANSWER	RANKS 2 TO 4
A (SST)	<code>scorpion</code> 0.54	<code>lion</code> 0.50 <code>lear</code> 0.50 <code>nuts</code> 0.49
A (text8)	<code>queen</code> 0.48	<code>infanta</code> 0.43 <code>hainault</code> 0.43 <code>forkbeard</code> 0.43
B (SST)	<code>sugar</code> 0.39	<code>abbott</code> 0.36 <code>ernest</code> 0.36 <code>wildcard</code> 0.35
B (text8)	<code>queen</code> 0.44	<code>kings</code> 0.39 <code>jehu</code> 0.37 <code>monarch</code> 0.34
C (SST)	<code>middle-aged</code> 0.71	<code>lion</code> 0.71 <code>creek</code> 0.70 <code>haynes</code> 0.70
C (text8)	<code>queen</code> 0.51	<code>kings</code> 0.42 <code>consort</code> 0.42 <code>isabella</code> 0.41
G	<code>queen</code> 0.71	<code>monarch</code> 0.62 <code>princess</code> 0.59 <code>crown_prince</code> 0.55
G-ft	<code>queen</code> 0.70	<code>princess</code> 0.60 <code>prince</code> 0.57 <code>queens</code> 0.56

actor - man + woman expected **actress**

MODEL	TOP ANSWER	RANKS 2 TO 4		
A (SST)	davies 0.47	macdowell 0.46	michel 0.46	stoop 0.46
A (text8)	actress 0.70	comedienne 0.54	singer 0.54	gingold 0.50
B (SST)	enigma 0.32	actress 0.32	unhinged 0.31	role 0.30
B (text8)	actress 0.69	novelist 0.46	musician 0.43	writer 0.43
C (SST)	succumb 0.65	academy 0.62	madonna 0.61	appearing 0.59
C (text8)	actress 0.71	comedienne 0.55	dancer 0.54	socialite 0.53
G	actress 0.86	actresses 0.66	thesp 0.63	Actress 0.62
G-ft	actress 0.82	actresses 0.69	singer 0.55	actors 0.55

better – good + bad expected worse

MODEL	TOP ANSWER	RANKS 2 TO 4		
A (SST)	concert 0.37	antitrust 0.36	einstein 0.36	hungry-man 0.36
A (text8)	worse 0.48	smarter 0.42	mattered 0.40	anemic 0.40
B (SST)	i.q. 0.32	franchise 0.29	further 0.29	funnier 0.28
B (text8)	more 0.41	less 0.39	fewer 0.38	tougher 0.37
C (SST)	enacted 0.71	badder 0.69	videotape 0.69	runner 0.69
C (text8)	worse 0.54	happier 0.46	slower 0.46	smarter 0.45
G	worse 0.73	uglier 0.57	sooner 0.53	dumber 0.52
G-ft	worse 0.66	uglier 0.55	differently 0.53	scarier 0.51

worst – bad + good expected **best**

MODEL	TOP ANSWER	RANKS 2 TO 4		
A (SST)	moronic 0.36	drowns 0.36	resources 0.36	bloated 0.35
A (text8)	best 0.45	razzie 0.42	evan 0.41	saddest 0.40
B (SST)	parents 0.31	teen-driven 0.28	made 0.28	shining 0.28
B (text8)	greatest 0.33	best 0.29	toughest 0.27	last 0.26
C (SST)	moronic 0.69	screenplays 0.69	simmer 0.69	sporting 0.66
C (text8)	best 0.45	greatest 0.44	razzie 0.44	evan 0.41
G	best 0.69	strongest 0.63	greatest 0.57	weakest 0.57
G-ft	best 0.58	strongest 0.57	finest 0.56	highest 0.56

comedy – funny + scary expected **horror**

MODEL	TOP ANSWER	RANKS 2 TO 4		
A (SST)	school-age 0.42	zippy 0.42	quadrangle 0.41	dumb 0.41
A (text8)	docu 0.51	sitcoms 0.51	screwball 0.51	slapstick 0.51
B (SST)	flick 0.36	opens 0.28	drama 0.28	innovations 0.27
B (text8)	screwball 0.33	comedies 0.32	scene 0.31	mgm 0.31
C (SST)	gross-out 0.76	psychotic 0.76	pandering 0.75	aspires 0.74
C (text8)	screwball 0.63	drama 0.63	comedies 0.62	stardom 0.61
G	horror_flick 0.55	horror 0.51	horror_thriller 0.51	terrifying 0.51
G-ft	gross-out 0.53	frightening 0.50	scarier 0.50	dramas 0.50

Every model trained on enough text solves the canonical analogy. G, G-ft and the three text8 models all rank **queen** first for **king – man + woman**. Trained on SST alone, A, B and C return unrelated words.

Film vocabulary does not rescue the SST-trained models. A, B and C trained on SST miss the film queries too. The one near hit is B (SST), which ranks **actress** second for **actor – man + woman**. G and G-ft put the expected word first for the actor, better and worst queries. The text8 models do better but not uniformly: all three rank **actress** first, A (text8) and C (text8) also solve the better and worst queries, and B (text8) solves neither. For **comedy – funny + scary**, G returns **horror_flick** and **horror**, and G-ft returns **gross-out** and **frightening**. The SST-only models fail even on their own vocabulary.

WHY A, B AND C FAIL

An analogy works when the offset between two words, such as **man** to **woman**, points the same way for other pairs, such as **king** to **queen** and **actor** to **actress**. A model learns a shared direction like that only after it sees each word in many different contexts. In the 6,920 training sentences, the expected answers appear between 11 and 131 times, and even

the most common query word appears only 214 times. The table gives each count next to the count in the phrase rows the models were trained on.

QUERY	OCCURRENCES: SENTENCES / PHRASE ROWS			
king - man + woman	king 13 / 50	man 65 / 313	woman 29 / 153	queen 11 / 43
actor - man + woman	actor 26 / 163	man 65 / 313	woman 29 / 153	actress 16 / 89
better - good + bad	better 108 / 532	good 214 / 1341	bad 149 / 907	worse 17 / 122
worst - bad + good	worst 40 / 271	bad 149 / 907	good 214 / 1341	best 131 / 807
comedy - funny + scary	comedy 212 / 1053	funny 181 / 1105	scary 19 / 97	horror 36 / 187

The phrase counts overstate what the models learned from. Most phrase rows are fragments of those same sentences (see [the data section](#)), so `queen` appears 43 times but in only 11 places in the original text. GoogleNews was trained on about 100 billion tokens, more than 150,000 times the 633,720 tokens here. The similarity benchmarks agree: on WordSim-353, A scores +0.029, B -0.036 and C +0.005, against G's +0.700.

Retraining A, B and C on text8 confirms the explanation. With 17,005,207 tokens and the same hyperparameters, all three answer `queen`, and their analogy accuracy rises to 0.372, 0.314 and 0.353. See [More training text](#).

Sentiment classification holds up anyway. A reaches 0.761 on SST-2 against G's 0.798. A classifier over averaged vectors only needs positive and negative words to separate roughly. That is a much coarser property than consistent analogy offsets, so the two evaluations measure different things.

MODEL	ANALOGY ACCURACY	QUESTIONS ATTEMPTED	COVERAGE	CANDIDATE POOL
A (SST)	0.0006	3,563 of 19,544	18.2%	top 50,000
A (text8)	0.3721	15,790 of 19,544	80.8%	top 50,000
B (SST)	0.0020	3,563 of 19,544	18.2%	top 50,000
B (text8)	0.3142	15,790 of 19,544	80.8%	top 50,000
C (SST)	0.0022	3,563 of 19,544	18.2%	top 50,000
C (text8)	0.3535	15,790 of 19,544	80.8%	top 50,000
G	0.7561	15,434 of 19,544	79.0%	top 50,000
G-ft	0.6750	3,563 of 19,544	18.2%	top 50,000

Accuracy needs to be read alongside coverage. gensim drops any question with an out-of-vocabulary word, so the SST-trained models are scored on only 18.2% of the set against G's 79.0%. G-ft's 0.675 comes from structure inherited through the warm start, not learned from SST.

SUPPORTING ANALYSIS, BEYOND THE REQUIREMENT

Good at sentiment, useless at similarity

The sections above cover the required sentiment task. This extra analysis is included because sentiment accuracy alone makes the embeddings look more capable than they are.

The three benchmarks compare cosine similarity with human ratings using Spearman correlation. Coverage is the fraction of pairs where both words are in the vocabulary. It is part of the result: the SST-trained models cover only about half of WordSim-353 because many benchmark words never occur in the movie reviews.

MODEL	WORDSIM-353	SIMLEX-999	MEN-3K	COVERAGE	SST-2 ACCURACY
A (SST)	+0.029	+0.045	+0.118	50.1%	0.761
A (text8)	+0.694	+0.341	+0.645	94.3%	0.722
B (SST)	-0.036	+0.069	-0.010	50.1%	0.729
B (text8)	+0.666	+0.259	+0.606	94.3%	0.724
C (SST)	+0.005	+0.033	+0.096	50.1%	0.744
C (text8)	+0.644	+0.332	+0.567	94.3%	0.715
G	+0.700	+0.442	+0.771	100.0%	0.798
G-ft	+0.585	+0.405	+0.702	50.1%	0.792

Trained on SST, A, B and C show no measurable lexical similarity structure. B is slightly negative on two benchmarks, which is effectively no correlation. Yet those same three models score 0.729 to 0.761 on sentiment, within 0.069 of GoogleNews. Mean-pooled logistic regression needs only that sentiment-bearing words point in consistent directions; it does not need `tiger` to sit near `cat`. Downstream accuracy by itself would make these embeddings look nearly as good as GoogleNews, which the intrinsic tests clearly contradict. The text8 rows are the same three models trained on more text, and they score close to GoogleNews; [More training text](#) covers them.

This also clarifies what the warm start contributes. G-ft scores 0.585 on WordSim-353 against 0.029 for A, trained from scratch on the identical corpus with identical settings. The pretrained initialization does more than add 0.031 accuracy. It supplies all of Model G-ft's measurable lexical structure.

BEYOND THE REQUIREMENT

More text fixes similarity, not sentiment

The sections above blame the size of the training data for the failures of A, B and C on similarity and analogies. To test that directly, I retrained all three on text8, the first 100 MB of cleaned English Wikipedia (mattmahoney.net/dc/text8.zip). It has 17,005,207 tokens, 27 times the SST phrase corpus. Every hyperparameter matches the SST runs. One implementation detail differs: C's text8 run uses sparse Adam, which updates only the rows a batch touches, because dense Adam is too slow at this vocabulary size (see [Cost to run](#)). C (text8) also trained on a PACE ICE A100 GPU, so its training time is not comparable with the laptop times. The sentiment test is also unchanged: the classifier is still fit on the SST phrase labels and scored on the same SST-2 test sentences, so only the word vectors differ.

MODEL	TOKENS	WORDSIM	ANALOGIES	KING - MAN + WOMAN	SST-2 ACC.	OOV
A (SST)	634k	+0.029	0.001	scorpion	0.761	6.6%
A (text8)	17M	+0.694	0.372	queen	0.722	16.9%
B (SST)	634k	-0.036	0.002	sugar	0.729	6.6%
B (text8)	17M	+0.666	0.314	queen	0.724	16.9%
C (SST)	634k	+0.005	0.002	middle-aged	0.744	6.6%
C (text8)	17M	+0.644	0.353	queen	0.715	16.9%
G	100B	+0.700	0.756	queen	0.798	27.1%

More text fixes the structure. WordSim-353 rises from +0.029 to +0.694 for A, from -0.036 to +0.666 for B and from +0.005 to +0.644 for C, close to GoogleNews at +0.700. Analogy accuracy goes from almost zero to 0.372, 0.314 and 0.353, and all three now answer `queen` for `king - man + woman`. The [analogy tables](#) and [neighbour lists](#) include the text8 models as extra rows.

Sentiment accuracy does not improve. A falls from 0.761 to 0.722, B moves from 0.729 to 0.724, and C falls from 0.744 to 0.715. The text8 vocabulary misses 16.9% of the SST-2 test tokens, against 6.6% for the SST models. 64% of the missing tokens are punctuation, which carries no sentiment. The rest matters more. text8 has no apostrophes, so `n't` is missing, and it occurs in 129 of the 1,821 test sentences. Review vocabulary is missing too: `refreshingly`, `soggy`, `mesmerizing`, `sappy`, `clunker`, `heartwarming` are all in the SST vocabulary but not in text8's, and several of them are strong sentiment words. Wikipedia teaches general word meaning. SST teaches the words reviewers use to praise and criticize, and the sentiment test rewards that.

TESTING THE HANDOUT'S CLAIMS

Page 1 of the handout makes two claims about the architectures that the two corpora can test. **CBOW trains several times faster.** On text8, B took 177 seconds against A's 889, a 5.0× difference (963,171 against 191,213 words per second). The claim holds. **Skip-gram does better on small data, and CBOW needs large data.** On SST, skip-gram leads CBOW by 0.015 in sentiment accuracy averaged over three seeds, and neither has any similarity structure. On text8, the accuracy gap disappears (+0.002 for B), while A still leads on WordSim-353 by 0.028 and on analogies by 0.058. CBOW closes the gap as the data grows, which fits the claim, but skip-gram stays ahead on the finer-grained tests. The text8 figures are single runs. On SST, the [hyperparameter sweep](#) puts the seed-to-seed standard

deviation of accuracy at 0.002 to 0.007, so the text8 accuracy gap is within noise. The WordSim-353 and analogy gaps are far larger than that.

MODEL G-FT

What fine-tuning means for Word2Vec

Fine-tuning Word2Vec is much simpler than fine-tuning a transformer. There are no frozen layers or adapters. I initialize `W_in` with published vectors, then continue ordinary skip-gram training on movie reviews at a low learning rate.

12,032

rows seeded from GoogleNews

2,277

rows Google's vocabulary lacks

0.961

mean cosine before and after training

+0.031

accuracy over training from scratch

MOVED MOST

kidman		0.553
philippe		0.599
schwarzenegger		0.608
holm		0.647
griffiths		0.663
sheridan		0.669
evelyn		0.677
clooney		0.700

DID NOT MOVE AT ALL

oozes		1.000
priggish		1.000
plod		1.000
worshipful		1.000
drudgery		1.000
abysmally		1.000
poo		1.000
execrable		1.000

The drift ranking mostly tracks corpus frequency. The words that move most are actor surnames, which are common in reviews but rare in news. At the other end, rare sentiment adjectives remain at cosine 1.000 because one or two SST appearances do not produce enough gradient to move them. The full ranking gives a clearer account of fine-tuning than one hand-picked word would.

01 Build the vocabulary from our corpus first

`intersect_word2vec_format` only overwrites rows that already exist. It does not import Google's three million words, so G-ft keeps the vocabulary built from SST.

02 Set `vectors_lockf` to ones explicitly

gensim 4.x does not create it. Without it the import either fails or silently freezes every row, leaving the imported vectors unchanged.

03 Keep the learning rate low: alpha 0.005, 8 epochs

Google never published their output matrix, so `W_out` starts random. Early updates must learn a new decoder as well as adjust the imported vectors. A standard learning rate distorts those vectors during the first epoch.

04 Measure drift rather than assuming it

Cosine between each word's warm-start vector and its final vector. The mean is 0.9609 and the median is 0.9731, confirming that the low learning rate preserved most of the pretrained structure.

MODEL C, BEYOND THE REQUIREMENT

Writing skip-gram with negative sampling

The handout suggests a brief look at the implementation. I chose to write it instead because that makes each part of Word2Vec concrete. `src/sgns.py` implements the vocabulary and counts, subsampling of frequent words, the $\text{unigram}^{0.75}$ noise table, a dynamic window, the two embedding matrices, and linear learning-rate decay. `to_keyedvectors` wraps `W_in` in a gensim object, which lets C use the same evaluation code as every other model.

	A, GENSIM	C, FROM SCRATCH
SST-2 accuracy	0.7611	0.7441
Wall clock	11.8 s	160.2 s
Words per second	537,887	39,571
Loss, first to last epoch	not exposed	4.2288 to 2.8681

The first version did not learn. Its loss stayed at exactly 1.3863 for all ten epochs, and accuracy was 0.4992, or chance on a balanced split. A loss of 1.3863 is $2 \cdot \ln 2$, what we get when every logit is zero. The problem was the loss reduction, not the sampler. Calling `BCEWithLogitsLoss` once on the positives and once on the negatives averages over B entries and over B·k entries, so at a batch of 8,192 the effective per-pair learning rate was about 3e-6. The measured gradient norm on `W_in` was exactly 0.0. I fixed it by writing Mikolov's equation 4 directly, summing across the k negatives, and averaging only across the batch.

Two component checks made the failure quick to locate. Subsampling retains 17.6% of occurrences of `the`. The noise table maps `the` from a raw frequency of 4.30% down to a sampled 1.31% while lifting `execrable` from effectively zero to 2e-5. Both results match the expected effect of the 0.75 exponent.

DELIVERABLE 4

What these models cost to run

Query latency measures brute-force `most_similar`, which runs a dense matrix-vector product over the full vocabulary. Each value comes from 200 calls. Memory is the RSS increase in an isolated subprocess, compared with the size of the matrices the model retains.

MODEL	VOCAB	P50 MS	P95 MS	KEEPS	PREDICTED MB	MEASURED MB	RATIO
A (SST)	14,309	0.41	2.41	2	34.3	35.0	1.02
B (SST)	14,309	0.36	1.40	2	34.3	48.0	1.40
C (SST)	14,309	0.37	1.51	1	17.2	17.8	1.03
G	500,000	15.08	26.94	1	600.0	195.6	0.33
G-ft	14,309	0.41	3.07	2	34.3	34.8	1.01

Brute-force query time grows with vocabulary size. G has 35 times as many words as the SST models and is 37 times slower at p50. An approximate index adds little value at 14,309 words. At 500,000, a query already takes 15 ms, and the full three-million-word Google model would take roughly 90 ms.

The `2 · V · d · 4` prediction holds within 3% for A, G-ft and C. B comes in at 1.40 because hierarchical softmax also keeps Huffman-tree code and point arrays that the formula omits.

G's memory result does not match the prediction. 500,000 by 300 float32 is 600 MB and the measured increase is only 195.6 MB. macOS memory compression is a possible explanation because it can compress inactive anonymous pages, causing `ru_maxrss` to fall below the allocation. A Linux run should land near 600 MB if that explanation is correct. Until then, the gap remains unresolved.

A separate benchmark determined how C trains. On this vocabulary, MPS is slower than CPU (64.3 against 44.6 ms per batch) and sparse gradients are slower than dense gradients (52.6 against 44.6). The model is small enough for GPU launch and data transfer overhead to dominate. Sparse updates also offer little benefit. A batch of 8,192 pairs with 10 negatives touches roughly 90,000 index slots across 14,309 rows, so almost every row updates on each step.

On text8, both results reverse. The vocabulary grows to 135,335 rows, and a dense Adam step has to update every one of them. A benchmark of a single training step at that size gave 1,300 ms per batch for dense updates on the CPU, against 207 ms for sparse updates on the CPU and 196 ms for dense updates on MPS. C's text8 run therefore uses sparse Adam. On the laptop it measured about 0.15 seconds per step, which puts the full 10-epoch run at roughly four hours, so it moved to a PACE ICE A100 GPU. There it trained in 659 seconds, 258,169 words per second. The GPU changed the speed and not the result: the laptop run, stopped after two epochs, had reached the same loss, 2.8402, at the end of epoch 2.

DELIVERABLE 2C

Why every model struggles with the same sentences

MODEL	ERRORS / 1,821	NEGATION	CONTRAST	OTHER
G	368	102	64	202
G-ft	379	96	64	219
A (SST)	435	101	72	262
C (SST)	466	114	79	273
B (SST)	493	120	62	311

Negation and contrast account for 37% to 45% of each main model's errors, against 32% of the test set. They are over-represented for every model, and most of all for GoogleNews (45%), so better embeddings do not solve it. Mean pooling removes word order, including the information needed to determine the scope of a negation.

DELIVERABLE 2C, STATED EXPLICITLY FOR MODEL G-FT

The handout asks for at least two test examples the model classifies correctly and at least two it gets wrong, for a named model. Model G-ft, the fine-tuned one, is used here. All four come from the same 1,821-sentence test split. Within each outcome, these are the highest-confidence cases.

OUTCOME	GOLD	PREDICTED	CONFIDENCE	SENTENCE
correct	positive	positive	0.999	an intelligent , moving and invigorating film .
correct	positive	positive	0.998	a smart , sweet and playful romantic comedy .
incorrect	negative	positive	0.952	it 's a great deal of sizzle and very little steak .
incorrect	negative	positive	0.938	a well acted and well intentioned snoozer .

CONFIDENT FAILURES ACROSS MODELS

MODEL	SENTENCE	GOLD	PREDICTED	CONFIDENCE	CATEGORY
A (SST)	not a bad journey at all .	pos	neg	0.958	negation
A (SST)	an ultra-low-budget indie debut that smacks more of good intentions than talent .	neg	pos	0.917	other
A (SST)	a well acted and well intentioned snoozers .	neg	pos	0.898	other
G-ft	never -lr- sinks -rr- into exploitation .	pos	neg	0.990	negation
G-ft	there is n't a weak or careless performance amongst them .	pos	neg	0.973	negation
G-ft	despite the long running time , the pace never feels slack -- there 's no scene that scr	pos	neg	0.955	negation

The failures follow directly from the representation. `not a bad journey at all` averages `not` and `bad` into negative territory. `there is n't a weak or careless performance amongst them` pools two strongly negative adjectives without preserving the negation that reverses them. In `a great deal of sizzle and very little steak`, the negative meaning comes from the contrast between clauses, which averaging also removes. The handout says to make a test set harder if it produces no failures. SST already contains clear failures, so no extra examples were needed.

DELIVERABLE 6

Installation, running and measurement

Four setup and measurement problems took most of the debugging time. In each case, the visible error pointed away from the actual cause. The symptoms and fixes are recorded here as part of the requested implementation experience.

01 gensim has no wheels for Python 3.14

The machine's default interpreter is 3.14. Installing gensim there starts a Cython source build and fails late in the process. The virtual environment therefore uses Homebrew Python 3.11.

02 `import gensim` fails on a fresh unpinned install

gensim 4.3.x imports `scipy.linalg.triu`, which scipy 1.13 removed. The `ImportError` points at gensim even though the incompatible scipy version is the cause. `requirements.txt` pins `scipy<1.13` and `numpy<2`.

03 The SST-2 split with hidden labels

The GLUE copy sets every test label to -1. An accuracy calculation against those placeholders can still produce a plausible number. `src/data.py` prevents that by requiring two classes in the test labels.

04 Whole-process peak RSS is not a model measurement

The first memory table compared process peak RSS against the matrix prediction and returned 297.8 MB for a model with 34.3 MB of matrices. Process peak RSS also includes the interpreter, gensim, numpy, torch, and the resident corpus. `src/measure_mem.py` now measures an RSS delta in an isolated subprocess per model.

After `pip install -r requirements.txt`, these commands produce the results the page is built from, in order:

01 `python -m src.data`

corpus statistics

02 `python -m src.main`

trains A and B on SST and loads G

03 `python -m src.run_e`

trains C on SST, 160 seconds on CPU

04

```
python -m src.run_eval
```

,

```
python -m src.measure_mem
```

,

```
python -m src.run_analogies
```

,

```
python -m src.testcases
```

similarity, latency, memory, errors, analogies and the worked test examples

05

```
python -m src.run_text8
```

downloads text8 and retrains A and B on it, about 18 minutes

06

```
python -m src.run_e --text8
```

trains C on text8. On a GPU cluster, `sbatch slurm/c_text8.sbatch` runs the same command; it saves a checkpoint after every epoch, so an interrupted job resumes where it stopped.

07

```
python -m src.sweep
```

the hyperparameter sweep

08

```
python -m src.build_report
```

rebuilds both report variants

Not everything has a command yet. G-ft, the A2 control and the PCA and t-SNE projections come from functions in `src/finetune.py`, `src/train_gensim.py` and `src/visualize.py` that have no command-line entry point. The deliverable 2b and 2c example files have no generating code in the repository at all. The list above therefore does not regenerate these; their results are stored in `results/`.

PROVENANCE

Build log

47fa014

HW2 phases 0-3

done

env, SST-2 data, gensim models A and B, sentiment pipeline

5cb380c	Phase 4 phrase-level SST corpus (4.7x text), Model C GoogleNews-300, OOV diagnostic	done
c86ba65	Phase 5 Model D, GoogleNews warm start fine-tuned on SST, with cosine drift analysis	done
340fe91	Phase 10 Model E, skip-gram with negative sampling from scratch in PyTorch	done
5551046	Phases 6-8 similarity benchmarks, latency and memory measurement, error mining	done
f8e08a2	Phase 9 generated HTML report in docs/, architecture SVGs, GitHub Pages layout	done
30690e9	Deliverable 3 five ranked test examples with retrieval; foreground sentiment in report	done
76a6f4a	Deliverables 2b and 6; fix misleading cosine field name; build PDF and submission zip Deliverables 2b and 6; fix misleading cosine field name; build PDF and submission zip	done
454d16c	Match HW1 deploy structure Vercel on public/, drop GitHub Pages, untrack instructor handout	done
675ec59	Address review PCA/t-SNE, analogies, explicit top-K, 2a and 2c, A2 control for the warm-start claim	done
ef49083	Split the overloaded chip highlight into two named marks; stop chips wrapping mid-token Split the overloaded chip highlight into two named marks; stop chips wrapping mid-token	done
40480f0	Split the write-up into a web variant and a coursework variant; add a sidebar contents Split the write-up into a web variant and a coursework variant; add a sidebar contents	done
78de142	Replace CLAUDE.md with AGENTS.md Replace CLAUDE.md with AGENTS.md	done

bdd7ecb	Humanize report and website copy Humanize report and website copy	done
ca1a4fb	Lead the data section with the corpus itself, not the pretrained baseline Lead the data section with the corpus itself, not the pretrained baseline	done
0e5127b	Show film-domain analogies next to king - man + woman Show film-domain analogies next to king - man + woman	done
00b6f8b	Rename model IDs E becomes C, C becomes G, D becomes G-ft	done
20a353e	Train A, B and C on text8; add a hyperparameter sweep and corpus statistics Train A, B and C on text8; add a hyperparameter sweep and corpus statistics	done
0132950	Report the text8 runs and the sweep; fix claims the new rows made false Report the text8 runs and the sweep; fix claims the new rows made false	done
960d14b	Rebuild the submission report so its provenance log lists the last three commits Rebuild the submission report so its provenance log lists the last three commits	done
1c97802	Retitle the report for two corpora and eight embeddings Retitle the report for two corpora and eight embeddings	done
pending	Hogwild scaling words/sec against worker count on a multi-core node	queued

Commit messages keep the model letters used when they were written. In them, E is the model now called C, C is now G, and D is now G-ft.

`src/build_report.py` reads every reported figure from `results/*.json` when it builds the page. Re-running a model and rebuilding therefore updates the prose and tables together.